07 - Referencia Técnica: APIs y Ejemplos

MCP Protocol - Cheat Sheet

CallToolRequest (Lo que Claude envía)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "tool_name",
    "arguments": {
      "param1": "value1",
      "param2": "value2"
    }
  }
}
```

CallToolResult (Lo que el servidor responde)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Operation succeeded"
      }
    ]
  }
}
```

Unity MCP - API Reference

Tool: create_script

Propósito: Crear un nuevo script C#

Parámetros:

```
{
  path: string;              // Ruta desde Assets/ (ej:
"Scripts/Player/Controller.cs")
  content: string;           // Código C# completo
  namespace?: string;        // Default: "ZoremGame"
  auto_compile?: boolean;    // Default: false
}
```

Ejemplo:

```
await mcp.callTool("create_script", {
  path: "Scripts/Combat/MeleeAttack.cs",
  content: "public class MeleeAttack { ... }",
  namespace: "ZoremGame.Combat",
  auto_compile: true
});
```

Tool: delete_script

Propósito: Eliminar un script

Parámetros:

```
{
  path: string;  // Ruta del script a eliminar
}
```

Tool: get_errors

Propósito: Obtener errores de compilación

Retorna:

```
{
  content: [
    {
      type: "text",
      text: "Compilation errors: ..." // o "No compilation errors"
    }
  ]
}
```

Tool: compile_project

Propósito: Forzar compilación

Parámetros: Ninguno

Tool: create_gameobject

Propósito: Crear un GameObject en una escena

Parámetros:

```
{
  name: string;
  scene_path: string;
  position?: [number, number, number];  // [x, y, z]
  parent?: string;  // Nombre del parent GameObject
}
```

Tool: configure_component

Propósito: Configurar un componente de un GameObject

Parámetros:

```
{
  gameobject_name: string;
  component_type: string;  // "Rigidbody", "BoxCollider", "Animator", etc.
  properties: {
    [key: string]: any;  // ej: {"mass": 80, "drag": 0.3}
  };
}
```

Ejemplo:

```
await mcp.callTool("configure_component", {
  gameobject_name: "Player",
  component_type: "Rigidbody",
  properties: {
    mass: 80,
    drag: 0.3,
    constraints: "FreezeRotationX|FreezeRotationZ"
  }
});
```

Blender MCP - API Reference

Tool: create_model

Propósito: Crear un modelo 3D

Parámetros:

```
{
  type: "humanoid" | "cube" | "sphere" | "custom";
  name: string;
  height?: number;
  width?: number;
  depth?: number;
  rigged?: boolean;
}
```

Ejemplo:

```
await mcp.callTool("create_model", {
  type: "humanoid",
  name: "Boss_Symbiote",
  height: 2.5,
  rigged: true
});
```

Tool: add_rig

Propósito: Agregar armadura (bones) a un modelo

Parámetros:

```
{
  model_name: string;
  rig_type: "humanoid" | "quadruped" | "custom";
  use_rigify?: boolean;  // Automatizar con Rigify
}
```

Tool: create_animation

Propósito: Crear una animación

Parámetros:

```
{
  model_name: string;
  animation_name: string;
  frame_start: number;
  frame_end: number;
  fps?: number;  // Default: 30
  keyframes?: Array<{
    frame: number;
```

```
    properties: { [key: string]: any };
  }>;
}
```

Tool: export_fbx

Propósito: Exportar modelo a FBX

Parámetros:

```
{
  blend_file: string;
  output_path: string;
  apply_modifiers?: boolean;
  optimize?: boolean;
  scale?: number;
}
```

Ejemplo:

```
await mcp.callTool("export_fbx", {
  blend_file: "/tmp/boss.blend",
  output_path: "/path/to/ZoremGame/Assets/Models/Boss_Symbiote.fbx",
  apply_modifiers: true,
  optimize: true
});
```

Tool: batch_export

Propósito: Exportar múltiples modelos

Parámetros:

```
{
  blend_files: string[];  // Array de paths
  output_folder: string;
  format: "fbx" | "glb" | "usdz";
}
```

Ejemplos de Prompts para Claude

Crear un Player Controller

```
Crea un PlayerController.cs que:
1. Mueva el personaje con WASD
2. Tenga un rigidbody configurado
3. Incluya comentarios explicativos

Ubicación: Assets/Scripts/Core/PlayerController.cs
Namespace: ZoremGame.Core
```

Crear un modelo de boss y exportarlo

```
Crea en Blender:
1. Modelo humanoid llamado "Boss_Fire"
2. Altura 3 metros
3. Agregar rig automático (Rigify)
4. Exportar a Assets/Models/Boss_Fire.fbx

Optimiza el modelo para juego en tiempo real.
```

Crear 4 personajes

```
Crea 4 modelos Blender diferentes:
1. Brute - humanoid musculoso, 2.2m
2. Agile - humanoid delgado, 1.8m
3. Mage - humanoid con capucha, 1.9m
4. Hybrid - mezcla de Brute y Mage

Agrégales rig a todos y exporta como FBX a Assets/Models/Characters/

Para cada uno, crea también un prefab en Unity con animator configurado.
```

Error Codes and Messages

Unity MCP

```
"File already exists: {path}"
└─ El archivo que intentas crear ya existe
└─ Solución: Cambiar nombre o usar force=true

"Scripts must be in Assets folder"
└─ Path está fuera de Assets/
└─ Solución: Usar ruta que empiece desde Assets/

"Unity project not found"
└─ ProjectSettings no existe en ruta
└─ Solución: Verificar UNITY_PROJECT_PATH en settings.json
```

```
"Compilation error: ..."
└─ El script tiene errores de C#
└─ Solución: Revisar el código generado
```

Blender MCP

```
"Blender not found"
└─ Blender executable no existe
└─ Solución: Verificar BLENDER_EXECUTABLE en settings.json

"Invalid model type"
└─ type no es uno de: humanoid, cube, sphere
└─ Solución: Usar tipos válidos

"Export failed: ..."
└─ Error durante exportación FBX
└─ Solución: Verificar que modelo es válido, revisar logs
```

Performance Benchmarks

Operaciones esperadas

Operación	Tiempo	Hardware
CreateScript	< 1s	Cualquiera
CompileProject	5-15s	Depende del proyecto
CreateModel (humanoid)	20-30s	Blender requiere startup
ExportFBX	10-20s	Blender optimiza modelo
CreateGameObject	< 1s	Cualquiera
ConfigureComponent	< 1s	Cualquiera

Notas:

- Primera llamada a Blender es lenta (startup)
- Blender en background es más rápido que interfaz GUI
- AutoCompile en Unity ralentiza operaciones

Archivos de Configuración

settings.json (Claude Code)

```
{
  "mcpServers": {
    "unity": {
      "command": "node",
      "args": ["/absolute/path/MCP/servers/unity/build/index.js"],
      "env": {
        "UNITY_PROJECT_PATH": "/absolute/path/ZoremGame",
        "UNITY_EXECUTABLE": "/path/to/Unity.app/Contents/MacOS/Unity",
        "ASSET_STORE_PATH": "Assets",
        "AUTO_COMPILE": "true"
      }
    }
  }
}
```

.env (Alternativa - si usas dotenv)

```
UNITY_PROJECT_PATH=/path/to/ZoremGame
UNITY_EXECUTABLE=/path/to/Unity
BLENDER_EXECUTABLE=/Applications/Blender/blender
```

Trucos y Tips

Optimizar velocidad

```
// Deshabilitar auto-compile durante batch operations
env: { AUTO_COMPILE: "false" }

// Luego compilar una vez al final:
await mcp.callTool("compile_project");
```

Debugging

```
// Habilitar verbose logging
console.error("[MCP]", "Tool:", request.params.name);
console.error("[MCP]", "Args:", request.params.arguments);
```

Batching operaciones

En lugar de:

```
createScript(...) → wait
createScript(...) → wait
createScript(...) → wait
```

Hacer:

```
Promise.all([
  createScript(...),
  createScript(...),
  createScript(...)
]) → wait all at once
```

Seguridad

Path Traversal Protection

```
// ❌ NUNCA permitir esto
const safePath = userInput.replace(/\.\.\//g, "");

// ❌ MEJOR - validar ruta completa
const fullPath = path.join(projectPath, safePath);
if (!fullPath.startsWith(projectPath)) {
  throw new Error("Invalid path");
}
```

Command Injection Protection

```
// ❌ NUNCA usar shell con input del usuario
exec(`blender ${userInput}`);

// ☑ MEJOR - usar array de argumentos
execFile("blender", ["--background", "--python", scriptPath]);
```

Contribuyendo

Para agregar nuevas tools:

1. Define el tipo en `src/types.ts`
2. Crea handler en `src/tools/`
3. Registra en `setRequestHandler`
4. Agrega test en `tests/`
5. Documenta en referencia técnica

Recursos Externos

- MCP Specification
- Unity Documentation
- Blender Python API
- Node.js Child Process

Fin de la documentación técnica.

Para empezar la implementación, ve a **README.md**.